Batched (CPU/GPU) entanglement pipeline for qubit networks: method, validation and comparison with QuTiP

1. What was implemented

For a graph G on n qubits, $H = \sum_{(i,j)\in E} h_{ij}$, every stage is vectorized over the whole time grid ($T = 5001$) and written against an array module, so the same code runs on NumPy and on CuPy (GPU).

1. **Evolution.** One eigendecomposition $H = VEV^\dagger$, then $\psi(t) = Ve^{-iEt}V^\dagger\psi_0$ for all t at once (exact; no ODE solver).
2. **Partial trace.** Reshape $\psi(t)$ to a $d_S \times d_{\bar S}$ matrix A_t; $\rho_S(t) = A_t A_t^\dagger$.
3. **Concurrence.** $C = \max(0, \lambda_1 - \lambda_2 - \lambda_3 - \lambda_4)$, λ_i the square roots of the eigenvalues of $\sqrt{\rho}\,\tilde\rho\sqrt{\rho}$, $\tilde\rho = (\sigma_y\otimes\sigma_y)\rho^*(\sigma_y\otimes\sigma_y)$ (Hermitian form: only batched `eigh`).
4. **PPT test and negativity bound.** Per cut M: $\lambda_{\min}(\rho^{T_M}) \geq 0$ (PPT $\Rightarrow$ GMN$= 0$); $N_M = \sum|\text{neg. eig}(\rho^{T_M})|$, GMN $\leq \min_M N_M$.
5. **Meyer–Wallach.** $Q = 2\big(1 - \frac{1}{n}\sum_i \operatorname{Tr}\rho_i^2\big)$.
6. **GMN (SDP)** for k-qubit subsets, $k = 3, 4, 5$: Secs. 2–3. (Guhne–Seevinck witness, C_{GME} and the negativity bound have no QuTiP counterpart and are not compared.)

2. GMN SDP, certificate and batched ADMM solver

For k qubits ($d = 2^k$) let $\mathcal{M}$ be the $K = 2^{k-1} - 1$ bipartitions ($K = 3, 7, 15$):

$$p^\star = \min_{W, P_M, Q_M} \operatorname{Re}\operatorname{Tr}(W\rho) \;\text{ s.t. }\; W = P_M + Q_M^{T_M},\; 0 \preceq P_M, Q_M \preceq I \;(M \in \mathcal{M}), \qquad \mathrm{GMN} = \max(0, -p^\star). \tag{1}$$

Dual certificate. Since $\min_{0\preceq P\preceq I}\operatorname{Tr}(\Lambda P) = \nu(\Lambda)$ (sum of negative eigenvalues), any Hermitian $\{\Lambda_M\}$ with $\sum_M \Lambda_M = \rho$ gives the rigorous bound GMN $\leq U(\Lambda) = \sum_M \big[|\nu(\Lambda_M)| + |\nu(\Lambda_M^{T_M})|\big]$.

ADMM. With $x = (W, \{P_M\}, \{Q_M\})$, $Ax = (P_M + Q_M^{T_M} - W)_M$, $c = (\rho, 0, \dots)$: as $T_M^2 = \mathrm{id}$, $AA^\dagger = 2I + \mathbf{1}\mathbf{1}^\top\otimes I$, so $(AA^\dagger)^{-1} = \frac{1}{2}(I - \frac{J}{2+K})$ and $\Pi_{\ker A}(x) = x - A^\dagger(AA^\dagger)^{-1}Ax$ needs only elementwise operations and partial transposes. W is unconstrained, hence $z_W = \hat x_W$ and its scaled dual is identically zero (not stored). With penalty ϱ, relaxation α, scaled dual u:

$$x^+ = \Pi_{\ker A}(z - u - c/\varrho), \qquad \hat x = \alpha x^+ + (1-\alpha)z, \tag{2}$$

$$z^+ = \Pi_{\mathcal{K}}(\hat x + u) \;\;(\text{clip eigenvalues of } P_M, Q_M \text{ to } [0,1]), \qquad u^+ = u + \hat x - z^+. \tag{3}$$

Certified stopping. The multiplier of the x-step, $\Lambda_M = \varrho y_M$ ($y = (AA^\dagger)^{-1}A(\cdot)$), satisfies $\sum_M \Lambda_M = \rho + \varrho(x_W - z_W)$, so after the Frobenius projection onto $\sum_M \Lambda_M = \rho$ it gives U. The primal estimate is $L = \max(0, -\operatorname{Re}\operatorname{Tr} W\rho)$. A state stops when $U - L \leq 10^{-7}$ and the primal residual $\leq 10^{-8}$; stopped states are removed from the batch. A batch of B states costs $2KB$ batched $d \times d$ `eigh`s per iteration. **Defaults:** $\varrho = 0.003$, $\alpha = 1.0$, 40000 iterations (tuned on hard states: Table 2).

Decision at threshold $\tau = 10^{-6}$: *zero* if $U \leq \tau$ (rigorous); *positive* if $L - (U - L) > \tau$ (margin rule, validated empirically, *not* rigorous since L is a primal estimate); otherwise *near threshold*, reported with the bracket $[L, U]$.

3. Comparison with QuTiP

ex$|+\rangle$ on the complete graph K_n, $T = 5001$, Tesla V100, one CPU core; seconds, GPU inputs device-resident. All implementations agree with the QuTiP versions to $\leq 2\times10^{-8}$ (concurrence), $\leq 10^{-10}$ (other stages).

Table 1 ($n = 5$)

Stage	ours NumPy	ours CuPy	QuTiP alternative	QuTiP / CuPy
Evolution	0.0099	0.0012	0.123 (`sesolve`); 34.0 (qutip-jax, GPU)	100×; 28000×
Partial trace, all pairs	0.035	0.0013	9.02 (`Qobj.ptrace` loop)	7000×
Concurrence, all pairs	3.96	0.019	8.46 (`qt.concurrence` loop)	440×
PPT min-eig, all triples, 3 cuts	1.27	0.052	11.6 (`partial_transpose` loop)	220×
Meyer–Wallach $Q(t)$	0.017	0.0017	5.00 (`ptrace` loop)	2900×

QuTiP times grow roughly linearly with n (per-state Python loops); CuPy barely changes (the $n = 3, 4$ columns are in the V100 results file in `gpu_bench`). `qutip-cuquantum` failed on the V100 (`CUTENSOR_ERROR`); on a T4 (weak FP64) CuPy is slower than NumPy for the small stages, so the GPU type matters.

GMN per solve (cvxpy+SCS, one core, vs GPU ADMM; QuTiP has no GMN solver). Production uses ∼40 SCS workers (≈17 ms per triple), so the GPU gain for triples is small; it is ∼30× for 4-qubit states, and the GPU is the only practical option for 5-qubit states. Times from an earlier fixed-iteration version.

	$k = 3$ (8×8)	$k = 4$ (16×16)	$k = 5$ (32×32)
cvxpy+SCS, 1 core	0.70 s	160 s	infeasible
GPU ADMM	0.053 s	0.135 s	1.1 s

4. Validation and design choices

Tuning (Table 2; K4, ex$|+\rangle$, converged = certified gap $\leq 10^{-7}$ within 40000 iterations): smaller ϱ and $\alpha = 1$ help the slowly converging states; $\varrho \geq 0.1$ converges almost none.

Table 2

Set	old (ϱ=0.03, α=1.5)	new (ϱ=0.003, α=1.0)
3-qubit subset (400 states)	95.5%	99.5%
4-qubit whole state (150 states)	67.3%	87.3%

Against CLARABEL. GHZ- and W-state sweeps through GMN = 0 ($k = 3$: 46 states each; $k = 4$: 6 of 46): no decision mismatches, max error 2.5×10^{-8} (GHZ_3), 1.3×10^{-8} (W_3), 9.6×10^{-9} (GHZ_4). On the 60 real states with the smallest negativity bound per case (GMN $\sim 10^{-8}$–10^{-5}), CLARABEL never exceeded U (6 settings, 45 comparisons), and no certified zero was wrong. The reference itself is unreliable there: every CLARABEL/SCS solve returned `optimal_inaccurate`, and one input gave values from 0 to 2.9×10^{-7} across tolerances/solvers, so the 10^{-7}–10^{-6} range cannot be arbitrated by the CPU solvers (this may also affect the SCS-based phase-1 results at the margin; not checked). L vs CLARABEL: below it for the old settings, above it by up to 8×10^{-7} (3-qubit) for the new ones; for 4-qubit near-threshold states by up to 10^{-5} in one case that was not resolved, and none of these converged. Hence the margin rule in Sec. 2.

Review of solver design (external suggestions, tested): dropping W_u is exact; the choice of multiplier estimate (y, u_P, averaged) makes no difference; the primal residual $\|x - z\|$ bounds the affine violation of z within a factor ≤ 3; masking instead of compaction is 60–80% slower; check intervals 25–500 change time by $< 10\%$; per-problem adaptive ϱ helps easy states by 3–10% and does not help the stuck ones (so it is off by default).